

Extending Computational Abstractions with Manual Craft for Visual Art Tools

Jingyi Li
Stanford University
Stanford, CA, USA
jingyili@cs.stanford.edu

ABSTRACT

Programming and computation are powerful tools for manipulating visual forms, but working with these abstractions is challenging for artists who are accustomed to direct manipulation and manual control. The goal of my research is to develop visual art tools that extend programmatic capabilities with manual craft. I do so by exposing computational abstractions as *transparent materials* that artists may *directly manipulate* and observe in a process that accommodates their *non-linear workflows*. Specifically, I conduct empirical research to identify challenges professional artists face when using existing software tools—as well as programming their own—to make art. I apply principles derived from these findings in two projects: an interactive programming environment that links code, numerical information, and program state to artists’ ongoing artworks, and an algorithm that automates the rigging of character clothing to bodies to allow for more flexible and customizable 2D character illustrations. Evaluating these interactions, my research promotes authoring tools that support arbitrary execution by adapting to the existing workflows of artists.

CCS CONCEPTS

- **Human-centered computing** → **Interactive systems and tools**;
- **Computing methodologies** → **Graphics systems and interfaces**.

ACM Reference Format:

Jingyi Li. 2022. Extending Computational Abstractions with Manual Craft for Visual Art Tools. In *The Adjunct Publication of the 35th Annual ACM Symposium on User Interface Software and Technology (UIST '22 Adjunct)*, October 29–November 2, 2022, Bend, OR, USA. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3526114.3558527>

1 INTRODUCTION

Visual art production increasingly makes use of software tools. Artists in traditionally non-digital fields like painting, photography, and animation now incorporate software tools (e.g., Adobe Photoshop) in their workflows. Like all media authoring tools, software offers new creative opportunities while also imposing constraints. For example, digital drawing software allows artists to continually edit their pieces with layers and undo, but limits them to predefined functionality provided by a toolbar and makes it difficult to integrate physical media. Developing software for artists presents

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST '22 Adjunct, October 29–November 2, 2022, Bend, OR, USA

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9321-8/22/10.

<https://doi.org/10.1145/3526114.3558527>

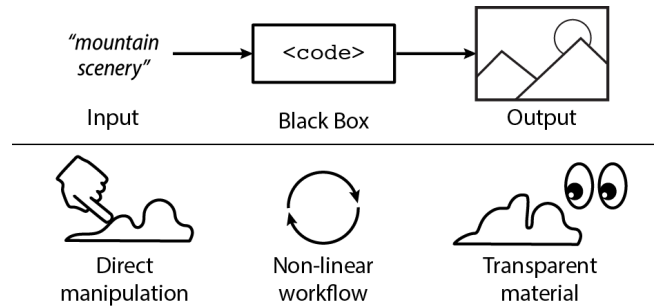


Figure 1: Compared to tools with rigid and opaque black box computational abstractions (top), my research (bottom) pushes for tools that allow users to directly manipulate computational structures, thus treating data abstractions as transparent materials that can be understood through non-linear workflows of action and observation.

challenges because their workflows are unlike those of typical programming. The highly structured nature of working with code can be incompatible with their existing practices of manually working with physical materials in an exploratory, idiosyncratic, and non-objective way [15].

Furthermore, domain-specific software abstractions may prevent artists from incorporating and adapting tools to their workflows. Abstractions—such as choosing which data and functionality to expose to the user and when—embed tool designers’ assumptions of how people will use their tools, which may not be aligned with the ways artists may want to use, or even appropriate, the tools (fig 1 top). As I present evidence for in section 2, “black box” models of software prevent artists from modifying, observing, and thus understanding the computational tools available to them [13].

Thus, the goal of my research is to create visual art authoring tools that extend the advantages of computation and programming with the advantages of manual craft. I hypothesize when a design tool’s computational structures (i.e., abstractions, representations, constraints) satisfy the following criteria (fig 1 bottom), they facilitate a creative process more aligned to that of manual craft:

- **Direct manipulation.** As opposed to manipulating a symbolic language (like writing code), artists should be able to visually manipulate concrete computational structures, to leverage their prior expertise of working with physical materials [1].
- **Transparent materials.** For a computational structure to be a transparent material, artists should be able to enact changes to it and immediately observe the results of the change. This allows artists to converse with, understand, and develop a relationship with the computational structure [3], similar to how they could engage with oil paint or chalk.

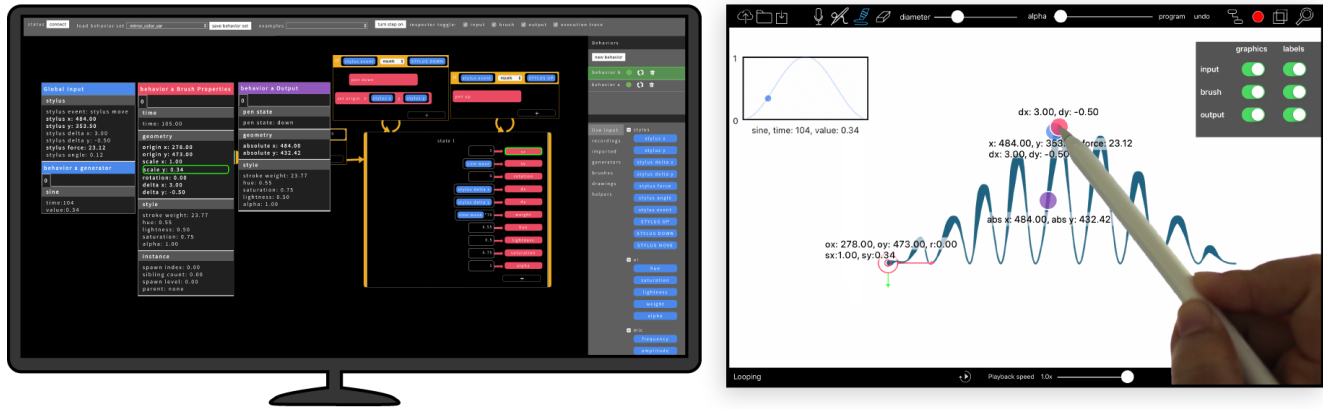


Figure 2: Demystified Dynamic Brushes (DDB) is a dual programming environment that links numerical inspectors in the programming environment (desktop, left) with visual inspectors in the drawing/execution environment (iPad, right).

- *Non-linear workflows.* Artists should be able to arbitrarily execute and modify computational structures at multiple points, rather than be tied to a rigid input-output process, to support their exploratory and experimental goals.

I make three contributions towards this goal. First, I analyze interviews with professional artists to understand the contexts and challenges of their current use, non-use, and personal development of digital tools [8]. Two key findings are that (i) artists learn to program to gain power, as (ii) automated software features (usually designed for novices) conflict with their goals of efficient manual practice and precise aesthetic control. To support (i) artists who program to make art, I design an interactive debugging environment that allows artists to inspect program state by directly touching their in progress artwork, transforming the process of programming closer to that of manual art creation [6]. I also contribute an automated algorithm that rigs accessories (such as clothing and hair) to the bodies of 2D mix-and-match characters (e.g., Bitmoji [14]), allowing for easier character customization while artists may still (ii) retain precise aesthetic control [9]. My ongoing work investigates how software abstractions may promote, rather than remove, the personal power and agency artists have over their tools. This leads to my future research agenda, where I hope to explore if these principles can also promote *inclusive* media authoring tools.

2 HOW ARTISTS USE SOFTWARE

To investigate the factors that lead artists to embrace or reject software tools created by others, as well as the factors that motivate artists to develop their own software, I conducted a thematic analysis of 13 interviews with professional artists across a diverse set of practices such as painting, typography, and interactive installation.

To summarize the findings, artists were motivated to develop their own software to create new functionality in their works, grow intellectually, and gain technical legitimacy and power in their communities. They selected software to use based on the available constraints fit in their workflows and low-level graphic representations—for instance, one artist spoke about how Bezier curves in vector graphics afforded meticulous editing of existing work, while bitmap representations pushed him to draw and move

on, as they did not support the same level of editing. All artists described going back and forth between integrating physical and digital media in their pieces, which was challenging given the current inflexibility of tools.

Another point of tension was when forms of software automation prevented artists from having full aesthetic control of their pieces. For example, one sculpture artist had to manually write her own vinyl cutting algorithm to interpret Bezier curves with some thickness as she could not modify the default algorithm on the machine and disliked how it handled intersections. Finally, artists found some of the most helpful software feature beyond producing artifacts (such as being able to save multiple versions of a file), and several—all women—initially resisted using computation because they did not share the values of existing technical communities, despite finding later success.

These findings imply critiques of existing approaches and strategies for new research opportunities. Current creativity support tools that seek to lower the threshold of creating art may rely on forms of automation and abstraction that hinder how artists traditionally learn through the *manual and direct manipulation of transparent materials*. Artists have complex and *non-linear workflows* that span physical and digital media, though an open challenge remains in unifying these representations to allow for more seamless and flexible transitions. Lastly, the fact that artists deliberately choose to use, reject, or build their own tools based on how the tool’s computational structures fit into their existing workflows shows they have a deep domain knowledge. As such, artists are suited to define domain-specific programming representations as technical collaborators; symmetrically, if systems engineering researchers wish to engage with artists, we should consider not only the tools we build, but how they impact their respective artistic communities.

3 INTEGRATING PROGRAMMING AND MANUAL CRAFT

As mentioned above, artists program not only to create new functionality in their artworks, but also to obtain intellectual growth and technical legitimacy in their communities. For instance, symbolic languages like Processing [12] have been widely adopted by

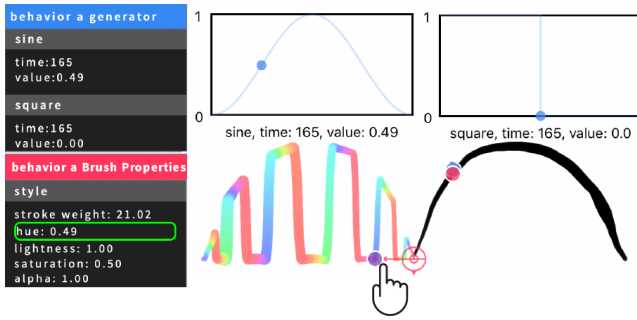


Figure 3: Using DDB’s direct-selection feature to understand the numerical values behind colors. Touching a point on a stroke jumps the program state to the point where the pixel was rendered.

the “creative coding” community. Although specialized for artistic applications, these languages impose the same workflows as general purpose programming languages and require understanding abstract representations and working in a highly structured manner. This poses a challenge for artists who are used to more iterative workflows where ideation and exploration are tightly coupled, and react to and plan code in the process of producing finished work. While artists are used to directly manipulating concrete visual forms, symbolic programming requires manipulating an abstract description of the work. I hypothesize that by providing visual artists with a creative coding environment that lets them *directly manipulate* their in-progress artwork to observe changes in program functionality and state, artists can engage with debugging as a *transparent material* to aid in their understanding of code.

To test this hypothesis, I built Demystified Dynamic Brushes (DDB, fig 2), an integrated inspection and execution control environment on top of the Dynamic Brushes [4] platform. DDB highlights numerical data values in the program by visualizing them on the drawing canvas. Artists can touch their in-progress artwork to highlight the corresponding numerical values in the code, and jump the program state to the line of code that drew the touched pixel (fig 3). DDB also lets artists loop their pen input data through the system while editing the symbolic program, which allows them to observe how code changes affect concrete artwork output at the speed of manual creation. The combination of DDB’s looping and direct selection features allows artists to rapidly transition from drawing to inspection (and vice-versa) at arbitrary points in the art creation process, supporting *non-linear workflows*.

An observational evaluation of DDB reveals that artists analyze programs when data is visualized in the drawing canvas, creating a closer link than displaying it numerically in the programming environment. Furthermore, artists preferred to debug at the speed and in style of manual drawing—to loop inputs and reasoning about how their code visually changed the artwork—over slowly stepping through each line in program execution.

4 AUTOMATED ACCESSORY RIGS

In addition to developing interactive environments, my research has also focused on making algorithmic contributions for visual art inputs more transparent. Specifically, in this project, I develop automated rigs—constraints between objects that propagate motion—for

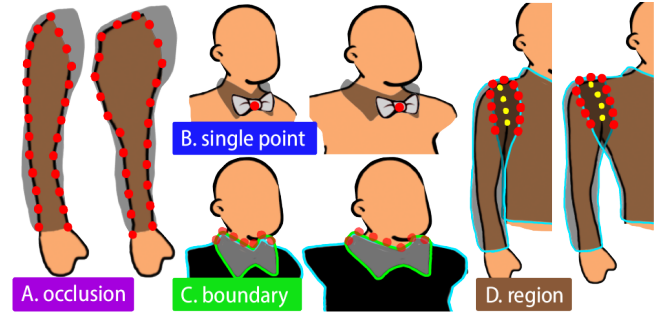


Figure 4: Our four different kinds of constraints between layers. For each group, the left is the input rig and the right is the rig under deformation.

2D illustrated mix-and-match characters. Mix-and-match character creation systems (e.g., Bitmoji [14]) allow users—artists and non-artists alike—to easily assemble different characters by compositing sets of predefined accessories like clothing or hair style. Each of these assets is represented as an interchangeable, independent artwork layer. However, these accessory layers are often designed to fit only the default body artwork, so modifying the body—such as changing its proportion or pose—means manually updating all the associated accessories as well.

To address this issue, I developed an automated rigging algorithm to capture and preserve important relationships between artwork layers so that the predefined accessories adapt with the character’s body, presented as an interactive design tool where users may *directly manipulate* (with handles as described in Bounded Biharmonic Weights [5]) and deform the rigged body. Four types of constraints that handle common interactions between layers encode these relationships: (1) occlusion, (2) attachment at a point, (3) coincident boundaries, and (4) overlapping regions (fig 4). The system takes as input a collection of layers, labeled as either belonging to the body or the accessories. It generates a directed acyclic graph to encode the parental relationships between layers to encode *which* layers should propagate deformations to which other ones based on layer overlap (e.g., the accessory sleeve layer overlaps with the arm, so any changes in the arm should affect the sleeve). In the case of an accessory layer overlapping multiple body layers, the system uses the z-ordering to construct an edge in the graph to the closest body layer to avoid cycles. Next, it infers which of the four constraint types to apply between layers in the graph: it applies occlusion constraints between most accessory-body connections, boundary and region constraints between two accessories with overlapping outlines or intersecting areas, and finally attachment at a single point for layers that do not satisfy the above. To support *non-linear workflows*, the tool gives artists the freedom to arbitrarily make body manipulations and swap accessories, enabling them to see how different body adjustments affect various accessories without having to redo their manipulations.

I evaluated this algorithm across a wide variety of input poses, accessories, and deformations. Some results from the OpenPeeps [10] dataset are shown in fig 5. While the rig inference algorithm offers reasonable defaults, the type of constraint applied to a layer does result in different aesthetics (fig 6). By exposing the rigs as a

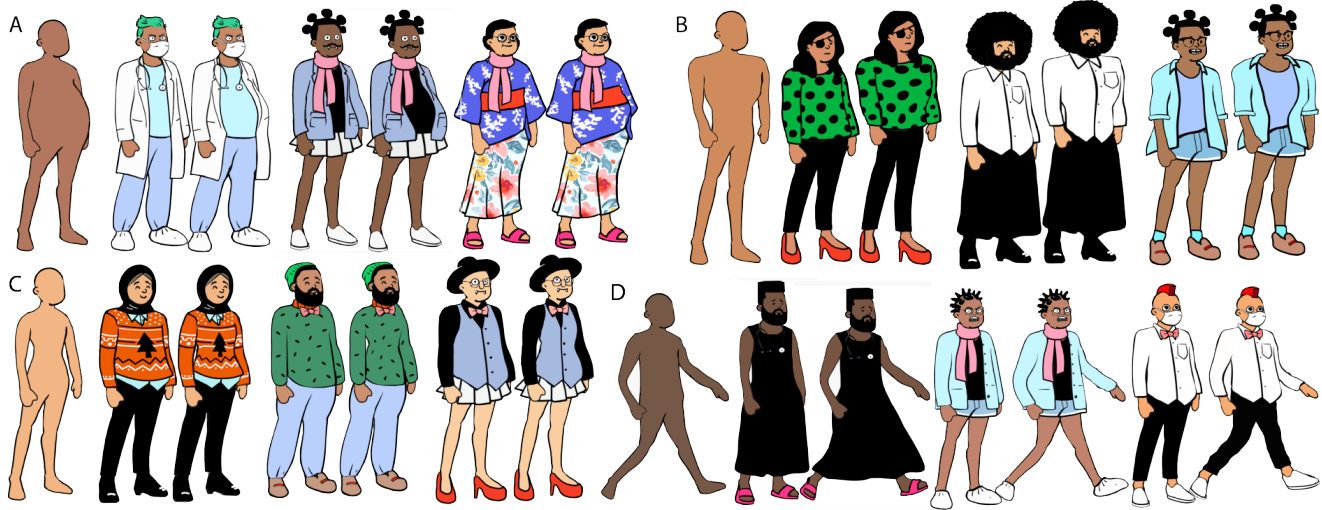


Figure 5: A sample of results from the Open Peeps dataset. For each pair of clothed results, the left is the input while the right is the deformed character.

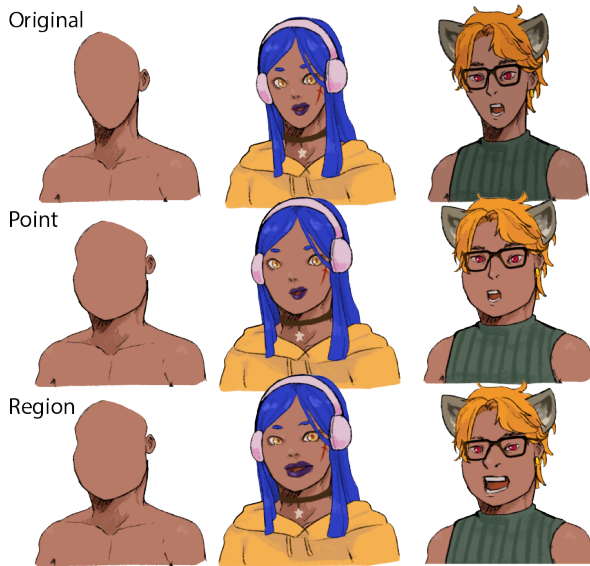


Figure 6: Results from widening the face of shoulder-up avatars that highlight how different constraint types (at a single point vs. overlapping region) result in different aesthetic outcomes.

transparent material and letting users easily change the inferred constraint types and even add or delete each constraint point, we enable expert artists to retain aesthetic control and give novices to rigging an opportunity to learn how the placement of rig points affect their artworks.

5 RESEARCH VISION

Going forward, my mission is to understand and develop *inclusive* media authoring tools. For instance, I collaborated with a blind web designer to develop a multi-modal tool that makes authoring web

page layouts accessible through translating the layouts to tactile printouts [7]: a tool that is inclusive of ability.

My past work has shown how accommodating manual inputs, malleable materials, and non-linear workflows can assist artists in better harnessing computational tools. While this research has focused on how tools can improve artifact creation, I want to examine how tools also *shift power* in the artistic communities they are meant to serve, such as how photography changed painting from realistic capture towards impressionism, or DALLÉ-2 [11] might reshape digital illustration. In *Tools for Conviviality* [2], Ivan Illich defines having power and agency over a tool as the “guarantee to each person’s right to the creative use of their energy.” Section 2 showed that black box software abstractions obscure data and processes and thus make it difficult to adapt tools into existing, idiosyncratic workflows. In my ongoing work, I hypothesize that abstractions in software tools also prevent their users from gaining personal *agency*. Through empirical observations and interviews, I seek to understand how software tools for visual art shift power across two dimensions: the user interface and design decisions of the tool itself, as well as the social contexts and communities of use surrounding the tool. For example, forms of automation may be more welcome in a professional labor context where an artist’s goal is to fulfill a client’s request as quickly as possible, but the same features may prevent a restorative, meditative experience with the tool when used in the context of a personal hobby. My eventual goal is to build flexible media authoring tools that work *with* humans rather than *for* them, towards a convivial future.

ACKNOWLEDGMENTS

Through my PhD I have met incredible people. I owe my advisors, collaborators, mentors, and colleagues for teaching me so much, and for helping me get this far.

REFERENCES

- [1] Edwin L Hutchins, James D Hollan, and Donald A Norman. 1985. Direct manipulation interfaces. *Human–computer interaction* 1, 4 (1985), 311–338.
- [2] Ivan Illich and Anne Lang. 1973. Tools for conviviality. (1973).
- [3] Tim Ingold. 2010. The textility of making. *Cambridge Journal of Economics* 34, 1 (2010), 91–102.
- [4] Jennifer Jacobs, Joel Brandt, Radomir Mech, and Mitchel Resnick. 2018. Extending Manual Drawing Practices with Artist-Centric Programming Tools. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (*CHI '18*). Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3173574.3174164>
- [5] Alec Jacobson, Ilya Baran, Jovan Popović, and Olga Sorkine. 2011. Bounded Biharmonic Weights for Real-Time Deformation. *ACM Trans. Graph.* 30, 4, Article 78 (July 2011), 8 pages. <https://doi.org/10.1145/2010324.1964973>
- [6] Jingyi Li, Joel Brandt, Radomir Mech, Maneesh Agrawala, and Jennifer Jacobs. 2020. *Supporting Visual Artists in Programming through Direct Inspection and Control of Program Execution*. Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3313831.3376765>
- [7] Jingyi Li, Son Kim, Joshua A. Miele, Maneesh Agrawala, and Sean Follmer. 2019. Editing Spatial Layouts through Tactile Templates for People with Visual Impairments. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) (*CHI '19*). Association for Computing Machinery, New York, NY, USA, 1–11. <https://doi.org/10.1145/3290605.3300436>
- [8] Jingyi Li, Wilmot Li, Sean Follmer, and Maneesh Agrawala. 2021. Automated Accessory Rigs for Layered 2D Character Illustrations. In *The 34th Annual ACM Symposium on User Interface Software and Technology* (Virtual Event, USA) (*UIST '21*). Association for Computing Machinery, New York, NY, USA, 1100–1108. <https://doi.org/10.1145/3472749.3474809>
- [9] Jingyi Li, Wilmot Li, Sean Follmer, and Maneesh Agrawala. 2021. Automated Accessory Rigs for Layered 2D Character Illustrations. In *The 34th Annual ACM Symposium on User Interface Software and Technology* (Virtual Event, USA) (*UIST '21*). Association for Computing Machinery, New York, NY, USA, 1100–1108. <https://doi.org/10.1145/3472749.3474809>
- [10] Pablo Stanley. 2020. *Open Peeps*. <https://www.openpeeps.com/>
- [11] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. 2022. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125* (2022).
- [12] C. Reas and B. Fry. 2004. *Processing*. <http://processing.org>.
- [13] Donald A Schon. 1968. The reflective practitioner. *New York* (1968).
- [14] Snap Inc. 2021. *Bitmoji*. <https://www.bitmoji.com/>
- [15] Bret Victor. 2011. *Dynamic Pictures*. <http://worrydream.com/DynamicPicturesMotivation>.